Assessment Tool:- CO1

String Cleaning

Name: Vishwa.T
Reg No: 192424180

Question

A customer database contains:

- Leading/trailing spaces
- Mixed uppercase and lowercase names
- Invalid email formats

Task:

- Clean names.
- Remove extra spaces.
- Validate email addresses using regular expressions.
- Display invalid records.

Code

```
import pandas as pd
import re

# Sample customer database with common data issues
data = {
    'CustomerID': [101, 102, 103, 104, 105],
    'Name': ['  john doe', 'MARY SMITH  ', 'raj KUMAR', '  Anita Rao  ',
'DAVID paul'],
    'Email': ['john.doe@gmail.com', 'mary_smith@@yahoo.com',
'rajkumar@.com',
              'anita.rao@company.co.in', 'davidpaulmail.com']
}

df = pd.DataFrame(data)

print("Original Data:")
print(df)
print("\n" + "="*60 + "\n")

# 1. Clean names: strip spaces, standardize to Title Case
df['Name_Cleaned'] = df['Name'].str.strip().str.title()

# 2. Remove extra internal spaces from names
df['Name_Cleaned'] = df['Name_Cleaned'].apply(lambda x: re.sub(r'\s+', ' ',
x))

# 3. Validate emails using regex
email_pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
df['Email_Valid'] = df['Email'].apply(lambda x: bool(re.match(email_pattern,
x.strip())))

print("Cleaned Data:")
```

```
print(df[['CustomerID', 'Name_Cleaned', 'Email', 'Email_Valid']])
print("\n" + "="*60 + "\n")

# 4. Display invalid records
invalid_records = df[~df['Email_Valid']]
print("Invalid Email Records:")
print(invalid_records[['CustomerID', 'Name_Cleaned', 'Email']])
```

Output (Google Colab Screenshot)

▶ String_Cleaning.ipynb

```
import pandas as pd
import re

# Sample customer database with common data issues
data = {
    'CustomerID': [101, 102, 103, 104, 105],
    'Name': ['  john doe', 'MARY SMITH  ', 'raj KUMAR', '  Anita Rao  ', 'DAVID paul'],
    'Email': ['john.doe@gmail.com', 'mary_smith@@yahoo.com', 'rajkumar@.com',
              'anita.rao@company.co.in', 'davidpaulmail.com']
}

df = pd.DataFrame(data)

print("Original Data:")
print(df)
print("\n" + "="*60 + "\n")

# 1. Clean names: strip spaces, standardize to Title Case
df['Name_Cleaned'] = df['Name'].str.strip().str.title()

# 2. Remove extra internal spaces from names
df['Name_Cleaned'] = df['Name_Cleaned'].apply(lambda x: re.sub(r'\s+', ' ', x))

# 3. Validate emails using regex
email_pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
df['Email_Valid'] = df['Email'].apply(lambda x: bool(re.match(email_pattern, x.strip())))

print("Cleaned Data:")
print(df[['CustomerID', 'Name_Cleaned', 'Email', 'Email_Valid']])
print("\n" + "="*60 + "\n")

# 4. Display invalid records
invalid_records = df[~df['Email_Valid']]
print("Invalid Email Records:")
print(invalid_records[['CustomerID', 'Name_Cleaned', 'Email']])
```

```
Original Data:
   CustomerID           Name                    Email
0         101       john doe       john.doe@gmail.com
1         102   MARY SMITH      mary_smith@@yahoo.com
2         103      raj KUMAR            rajkumar@.com
3         104    Anita Rao    anita.rao@company.co.in
4         105     DAVID paul        davidpaulmail.com

============================================================

Cleaned Data:
   CustomerID Name_Cleaned                    Email  Email_Valid
0         101     John Doe       john.doe@gmail.com         True
1         102   Mary Smith    mary_smith@@yahoo.com        False
2         103    Raj Kumar            rajkumar@.com        False
3         104    Anita Rao  anita.rao@company.co.in         True
4         105   David Paul        davidpaulmail.com        False

============================================================

Invalid Email Records:
   CustomerID Name_Cleaned                 Email
1         102   Mary Smith  mary_smith@@yahoo.com
2         103    Raj Kumar          rajkumar@.com
4         105   David Paul      davidpaulmail.com
```